

Subquadratic Sparse Attention: Content-Routed Exact Attention for Long Contexts

Jon Smirl

September 5, 2026

Abstract

Dense self-attention costs $O(n^2)$ in the sequence length n , which is the binding constraint on long-context language models. This paper presents *Subquadratic Sparse Attention* (SSA), an attention mechanism that, for each query, (i) routes to a small content-dependent set of key blocks using only per-block summary statistics, (ii) adds a local window, and (iii) performs *exact* softmax attention over the selected keys. The per-query work is $O(\kappa)$ in a fixed budget $\kappa \ll n$ plus a sublinear routing cost: a flat router runs the layer in $O(n\sqrt{n})$ (at block size $b = \sqrt{n}$, where the budget grows as $\sqrt{n}$), and a hierarchical router runs it near-linearly at fixed b and fixed κ —the configuration in which retrieval is also flat in n (Section 4.4 keeps the two regimes apart). A supporting theory establishes when this is sound. A retrieval-margin analysis shows that softmax attention recovers a target key with weight $\sigma(\beta\Delta - \log \mu)$, where Δ is the score margin and μ the number of competing distractors; selection works because it cuts μ from n to κ , making recovery *flat* in n . The analysis gives the admissible routing bound that licenses summary-only selection, a tempered (cumulant) routing score that—unlike centroid routing—sees in-block outliers, and a closed-form variance prune test from Samuelson’s inequality. A limit is then proved: cheap and lossless selection cannot hold for arbitrary keys (a one-line probe argument); adding length-robustness gives the trilemma (at most two of the three). So SSA’s subquadratic *exactness* is licensed by the *benign geometry* of trained representations—geometry that training can be made to manufacture, via a routability regularizer that shrinks off-target spread. Finally, length generalization (rotary position plus staged continued training reaches $32\times$ the trained length at 0.98 recall for ~ 800 adaptation steps) and a construction pipeline that converts a dense pretrained model into a subquadratic one by swapping the attention and briefly adapting (recovering to within +1.2 perplexity of a dense model given equal training while attending 38% of keys) are demonstrated. All claims are accompanied by measurements at controlled scale. A separate adaptive reference bounds omitted softmax mass, the subset-to-dense KL divergence, and attention-output error from key and value summaries, with a full-scan fallback when the requested tolerance cannot be certified sparsely.

1 Introduction

A transformer layer computes, for queries $Q \in \mathbb{R}^{n \times d}$, keys $K \in \mathbb{R}^{n \times d}$, and values $V \in \mathbb{R}^{n \times d}$,

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

The $n \times n$ score matrix makes both time and memory $\Theta(n^2d)$. For contexts of 10^6 – 10^7 tokens this is prohibitive, yet most of the matrix is near-zero: for a given query only a small set of keys carries appreciable weight. The question is whether one can *find* that set without forming all n^2 scores.

Two subquadratic families answer differently. *Kernel / linear attention* replaces $\exp(\langle q, k \rangle)$ by a factorizable feature map $\phi(q)^\top \phi(k)$, giving $O(n)$ cost but a low-rank (smoothed) attention matrix. *Sparse / selective attention* keeps the exact softmax but evaluates it only on a chosen subset of keys. SSA is in the second family, with three design commitments:

1. **Content-dependent selection.** The chosen keys depend on the query, not only on position; this is what lets a query reach the one relevant block a million tokens back.
2. **Summary-only routing.** Selection uses per-block statistics (a mean, a covariance), so it does not read every key—that is where the subquadratic saving comes from.
3. **Exact attention over the selected set.** Within the selected keys the softmax is computed exactly, so there is no kernel-approximation error on the keys that matter.

The paper is organized around one tension. Selection is cheap only if it can judge a block from its summary; it is *correct* only if those summaries do not hide a key that mattered. Sections 3–5 make both sides precise; Section 6 shows they cannot both hold in the worst case, so something must give; Section 7 shows what gives—the data geometry, which training shapes. Sections 8–9 turn the mechanism into a usable recipe.

2 Notation and the retrieval view

Fix a single query $q \in \mathbb{R}^d$ and keys $k_1, \dots, k_n$. Write the *logit* (score) of key j as

$$a_j = \langle q, k_j \rangle, \quad w_j = \frac{e^{\beta a_j}}{\sum_{l=1}^n e^{\beta a_l}}, \quad \beta = \frac{1}{\sqrt{d}},$$

so w is the attention distribution and β its inverse temperature. The output is $o = \sum_j w_j v_j$.

A long-context layer is, operationally, an *associative recall*: a query must place most of its weight on the key(s) that hold the information it needs and little on the rest. Attention is therefore analyzed by the weight it puts on a designated *target* key $k_\star$ relative to the *distractors* $\{k_j\}_{j \neq \star}$.

3 Retrieval margin and the recovery weight

Define the *margin* of the target as the gap between its logit and the typical distractor logit,

$$\Delta = a_\star - \bar{a}_{\text{dist}}, \quad \bar{a}_{\text{dist}} = (\text{typical } a_j, j \neq \star).$$

Suppose there are μ effective distractors with logit $\approx a_\star - \Delta$. Then the target weight is

$$w_\star = \frac{e^{\beta a_\star}}{e^{\beta a_\star} + \mu e^{\beta(a_\star - \Delta)}} = \frac{1}{1 + \mu e^{-\beta \Delta}} = \sigma(\beta \Delta - \log \mu), \quad (3.1)$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the logistic. Equation (3.1) is the *recovery weight*. Two consequences:

- **Detectability threshold.** The target is recovered ($w_\star > \frac{1}{2}$) iff

$$\boxed{\beta \Delta > \log \mu} \quad (3.2)$$

Recovery degrades only *logarithmically* in the number of distractors.

- **Why selection helps, and why it is flat in n .** Dense attention pays $\mu = n$. A selector that attends only a budget of κ keys pays $\mu = \kappa$, replacing the threshold $\log n$ by $\log \kappa$. If κ is held *fixed* as the context grows, the threshold (3.2) does not move with n : retrieval accuracy becomes *independent of context length*. This is the entire value proposition of selection, and it is why a fixed-budget selector can answer single-target queries at 10^6 – 10^7 tokens that dense attention, with its $\log n$ erosion, increasingly struggles with.

The catch is hidden in the phrase “a selector that attends κ keys”: the selector must *contain the target in its budget*. Sections 4–6 are about exactly that.

4 The algorithm

4.1 Block partition and summaries

Partition the n keys into B contiguous *blocks* of size b (so $Bb = n$). For block c precompute, once, the summary statistics

$$\mu_c = \frac{1}{b} \sum_{j \in c} k_j \quad (\text{mean}), \quad \Sigma_c = \frac{1}{b} \sum_{j \in c} (k_j - \mu_c)(k_j - \mu_c)^\top \quad (\text{covariance}), \quad (4.1)$$

and a radius $R_c = \max_{j \in c} \|k_j - \mu_c\|$. In practice Σ_c is kept diagonal, $\sigma_c^2 = \text{diag } \Sigma_c$, so a block’s summary is $O(d)$ numbers. (The diagonal form is fine for the *heuristic* routing score, but it voids the Samuelson prune *certificate* unless the surrogate bound is used—see the diagonal-summary caveat in Section 5.3.)

4.2 Routing score

For a query q , the *routing score* of block c estimates the largest logit the block can contribute. The exact quantity is the block’s log-sum-exp $\text{LSE}_c(q) = \log \sum_{j \in c} e^{\beta \langle q, k_j \rangle}$, which requires reading every key. SSA uses its *second-order (cumulant) surrogate*, computable from the summary alone:

$$r_c(q) = \langle q, \mu_c \rangle + \frac{\beta}{2} q^\top \Sigma_c q \approx \langle q, \mu_c \rangle + \frac{\beta}{2} \langle q^2, \sigma_c^2 \rangle. \quad (4.2)$$

Equation (4.2) is the Taylor/cumulant expansion of the tempered mean $\beta^{-1} \log \frac{1}{b} \sum_{j \in c} e^{\beta \langle q, k_j \rangle}$ about $\beta = 0$: the first cumulant is the mean logit $\langle q, \mu_c \rangle$; the second adds the *variance of the logit across the block*, $q^\top \Sigma_c q$. The variance term is what makes routing see an in-block outlier (Section 5.2).

4.3 Selection and exact attention

For each query q_i (at position i):

1. Compute $r_c(q_i)$ for all blocks c whose keys are causally visible (c ends at or before i).
2. Select the top- k blocks by r_c , *union* a local window of the w most recent blocks (recency is always relevant and cheap to include), giving a key set S_i with $|S_i| = \kappa \leq (k + w)b$.
3. Compute exact softmax attention restricted to S_i :

$$o_i = \sum_{j \in S_i} \frac{e^{\beta \langle q_i, k_j \rangle}}{\sum_{l \in S_i} e^{\beta \langle q_i, k_l \rangle}} v_j. \quad (4.3)$$

Algorithm 1: SSA forward (one head)

```

Input:  Q, K, V in  $\mathbb{R}^{n \times d}$ ;  block size  $b$ ;  budgets  $k$  (global),  $w$  (local)
Precompute: for each block  $c$ , mean  $\mu_c$  and diagonal covariance  $\Sigma_c$       //  $O(n d)$ 
For each query  $q_i$  (in parallel over  $i$ ):
  Routing:   for each causally visible block  $c$ :
               $r_c \leftarrow \langle q_i, \mu_c \rangle + (\beta/2) * \langle q_i^2, \text{diag } \Sigma_c \rangle$ 
  Selection:  $T \leftarrow (\text{top-}k \text{ blocks by } r_c) \cup (\text{w most-recent blocks})$ 
  Causal:    $S_i \leftarrow \text{keys of blocks in } T \text{ with position } \leq i$ 
  Attention:  $o_i \leftarrow \text{softmax}_{\{j \in S_i\}} ( \langle q_i, k_j \rangle / \sqrt{d} ) * V[S_i]$   // exact
Return 0

```

4.4 Complexity—two regimes, and which claim lives where

Routing is $O(n B d)$ if every query scores every block, and attention is $O(n \kappa d)$. Two parameter regimes must be kept apart, because the cost claim and the retrieval-flatness claim of Section 3 do not live in the same one:

- **Cost-optimal flat router**, $B = \sqrt{n}$, $b = \sqrt{n}$, fixed block-counts k, w : routing $O(n^{1.5}d)$, attention $O(n \kappa d) = O(n^{1.5}d)$; total $O(n^{1.5}d)$ —already a large saving over n^2 . But here the key budget $\kappa = (k + w)\sqrt{n}$ *grows with n* : the detectability threshold $\log \kappa$ of Section 3 rises as $\frac{1}{2} \log n$ (the $\log n$ erosion returns at half rate), and the $1/b$ mean attenuation of Section 5.2 worsens as $1/\sqrt{n}$. Subquadratic, but *not* retrieval-flat.
- **Retrieval-flat flat router**, b fixed (the measured setting, $b = 128$), fixed k, w : κ is fixed, so recovery is flat in n (Section 3)—but scan-all-blocks routing is $O(n B d) = O(n^2 d/b)$, quadratic up to a constant (the measured $n^{1.76}$ router and $n^{2.12}$ maskbuild walls of Section 10).
- **Hierarchical router at fixed b and fixed κ** —the configuration that delivers both. Organize blocks into a tree of summaries and descend it per query with branch-and-bound pruning (Section 5.1), so each query inspects $O(k \log B)$ nodes rather than all B —a benign-geometry cost, not a worst-case guarantee. Routing drops toward $O(n \log n d)$ while κ stays fixed. The practical instance is the *approximate* IVF router of Section 10 (0.93–0.97 block agreement), which is how the measured 12M-token forward reaches $\sim 2.9\times$ the $n \kappa$ floor—in the cheap+length-robust-but-approximate corner of Section 6, not the certified one.

So “ $O(n\sqrt{n})$ per layer” and “recovery flat in n ” are claims about *different* configurations, and the hierarchical (or IVF) router at fixed b, κ is what reconciles them. Either way the dominant n^2 term is gone. In a block-sparse kernel implementation the practical speedup over a dense exact kernel was $20.6\times$ at $n = 262,144$ on a single accelerator (Section 11).

5 Routing theory: when a summary is enough

The routing score (4.2) is a *heuristic* surrogate. This section gives the *certified* objects—upper bounds that make selection provably lossless—and the prune test SSA uses.¹

¹These supporting results—the admissible and ellipsoidal search bounds, the Samuelson prune gate, the tempered log-partition sandwich, the recursive-radius correctness of hierarchical pruning, the value-aware drop bound, and the no-free-selection limit of Section 6—are machine-checked in a Lean 4 formalization (axiom-pure, no `sorry`) [17], which is maintained separately and is not part of this repository’s artifact.

5.1 The admissible bound and branch-and-bound exactness

For any key k in block c , decompose its logit around the block mean and apply Cauchy–Schwarz:

$$\langle q, k \rangle = \langle q, \mu_c \rangle + \langle q, k - \mu_c \rangle \leq \langle q, \mu_c \rangle + \|q\| \|k - \mu_c\| \leq \underbrace{\langle q, \mu_c \rangle + \|q\| R_c}_{U_c(q)}. \quad (5.1)$$

$U_c(q)$ is an *admissible upper bound*: no key in block c has a logit exceeding it, and it depends only on the summary (μ_c, R_c) . This licenses exact selection at adaptive cost.

Branch-and-bound selection. Maintain the best *actual* logit found so far, s^* . Open blocks in decreasing $U_c(q)$. Stop as soon as the next block has $U_c(q) \leq s^*$: by (5.1) no unopened block can contain a key beating s^* . The result is identical to scanning all keys, but only the blocks whose bound clears s^* are ever read.

The cost of branch-and-bound is the number of blocks whose bound exceeds the true best—a quantity governed entirely by how *tight* the bound (5.1) is, i.e. by the geometry of the keys (Sections 6–7).

Anisotropic refinement. The isotropic radius R_c is loose when a block’s keys are spread unevenly across directions. Using the covariance ellipsoid instead,

$$\langle q, k - \mu_c \rangle \leq \sqrt{q^\top \Sigma_c q} \cdot \rho_c, \quad \rho_c = \max_{j \in c} \|\Sigma_c^{-1/2}(k_j - \mu_c)\|, \quad (5.2)$$

which replaces $\|q\| R_c$ by the *directional radius* $\sqrt{q^\top \Sigma_c q} \cdot \rho_c$. When the keys are anisotropic (the usual trained case), $\sqrt{q^\top \Sigma_c q}$ can be far smaller than $\|q\| R_c$ for queries pointing along thin directions—a tighter, query-specific bound, and exactly the quantity the cumulant score (4.2) already trades on.

The regularised form is not the same bound. Equation (5.2) is admissible in exact arithmetic, but Σ_c is singular whenever a block holds fewer keys than the head dimension—the usual case—so every implementation forms $S_c = \Sigma_c + \varepsilon I$ and takes $\rho_c = \max_j \|S_c^{-1/2}(k_j - \mu_c)\|$. *The quadratic form must then be S_c ’s as well*: Cauchy–Schwarz reads $\langle q, k - \mu_c \rangle = \langle S_c^{1/2} q, S_c^{-1/2}(k - \mu_c) \rangle \leq \sqrt{q^\top S_c q} \cdot \rho_c$, and $q^\top S_c q = q^\top \Sigma_c q + \varepsilon \|q\|^2$. Pairing a radius measured in the S_c metric with the *unregularised* $\sqrt{q^\top \Sigma_c q}$ yields a quantity that can fall below the block’s own maximum, and a bound that does so can discard the block holding the argmax—at which point losslessness is no longer a property of the construction. The deficit is small and direction-dependent: on 32-dimensional blocks of ≈ 33 keys it appeared on 5 of 128 block–query pairs, by at most 2.8×10^{-3} , and *only* for queries aligned with a block’s leading direction. That last clause is the practical warning, because it is the realistic case—a router’s queries are not orthogonal to

the keys they route to—and it is why a random-query admissibility test at a single geometry does not detect the error.

5.2 Why second order: centroid routing is blind to outliers

Centroid routing uses only $r_c = \langle q, \mu_c \rangle$. A single key k_* in a block of size b contributes $\frac{1}{b}k_*$ to μ_c , so its signal in the mean is attenuated by $1/b$: a lone target in a large block is invisible to centroid routing. The variance term in (4.2) repairs this, because an outlier aligned with q inflates $q^\top \Sigma_c q$. Concretely, if block c holds one key with $\langle q, k_* \rangle = a_*$ and $b - 1$ keys with logit ≈ 0 , then $\langle q, \mu_c \rangle \approx a_*/b$ but $q^\top \Sigma_c q \approx a_*^2/b$, so the second-order score $r_c \approx a_*/b + (\beta/2)a_*^2/b$ carries the quadratic outlier signal the mean discards.

The tempered family and its bias. It is the *normalized* tempered mean $g_c^{(\beta)} = \beta^{-1} \log \frac{1}{b} \sum_{j \in c} e^{\beta \langle q, k_j \rangle}$ —the object expanded in (4.2)/Appendix A.2—that interpolates between the mean logit ($\beta \rightarrow 0$) and the block max ($\beta \rightarrow \infty$). The unnormalized log-sum-exp $r_c^{(\beta)} = \beta^{-1} \log \sum_{j \in c} e^{\beta \langle q, k_j \rangle} = g_c^{(\beta)} + (\log b)/\beta$ differs from it only by a block-size constant (irrelevant when ranking equal-size blocks, where the two orderings coincide) and is sandwiched by

$$\max_{j \in c} \langle q, k_j \rangle \leq r_c^{(\beta)} \leq \max_{j \in c} \langle q, k_j \rangle + \frac{\log b}{\beta}. \quad (5.3)$$

So the tempered score estimates the block’s *best* logit—exactly what selection wants—with bias at most $(\log b)/\beta$. Small β smooths over outliers (the centroid failure); large β removes the bias but amplifies noise and loses the averaging that makes summaries stable. The cumulant form (4.2) is the second-order Taylor truncation of $g_c^{(\beta)}$ about $\beta = 0$, used at the measured optimum $\beta \approx 2$; its truncation error is the Lagrange remainder $(\beta^2/6) K_c'''(\xi)$ for some $\xi \in (0, \beta)$, with K_c the cumulant generating function of the in-block logits—a third-cumulant (skew) term that vanishes for light-tailed blocks and is *positive* for a positively-skewed (spiked) block, so the second-order score under-estimates exactly the outlier blocks it most needs to see at moderate β . That residual gap is the mechanism behind the isolated-needle failures measured in Section 11, and the reason the implementation offers an Edgeworth variant that adds the diagonal third-cumulant term.

5.3 The Samuelson prune test

A closed-form, summary-only test for *discarding* a block uses *Samuelson’s inequality*: for any reals $s_1, \dots, s_m$ with mean $\bar{s}$ and population variance $\text{Var} = \frac{1}{m} \sum_j (s_j - \bar{s})^2$, every element obeys

$$(s_i - \bar{s})^2 \leq (m - 1) \text{Var}, \quad \text{equivalently} \quad \max_j s_j \leq \bar{s} + \sqrt{(m - 1) \text{Var}}. \quad (5.4)$$

Apply it to the in-block logits $s_j = \langle q, k_j \rangle$, whose mean is $\langle q, \mu_c \rangle$ and whose variance is $q^\top \Sigma_c q$. Then the block’s best logit is bounded by $\langle q, \mu_c \rangle + \sqrt{(b - 1) q^\top \Sigma_c q}$, giving the *prune gate*: block

c can be safely discarded against a threshold $\tau = s^*$ whenever

$$\boxed{(s^* - \langle q, \mu_c \rangle)^2 > (b-1) q^\top \Sigma_c q \quad \text{and} \quad \langle q, \mu_c \rangle < s^*} \quad (5.5)$$

i.e. when the *margin* of the current best over the block mean exceeds $\sqrt{(b-1) \cdot \text{spread}}$. The test needs only (μ_c, Σ_c) . It is *sufficient* (it never wrongly prunes) but not necessary—though by Proposition 1 it is the *tightest* test computable from those statistics, so its non-necessity is a property of summary-only routing and not a slack in this particular gate. It sharpens the radius bound (5.1) when the block’s spread along q is small relative to its worst-case radius ($\sqrt{(b-1) q^\top \Sigma_c q} \ll \|q\| R_c$); neither bound dominates in general— $\sqrt{(b-1) q^\top \Sigma_c q}$ can exceed $\|q\| R_c$ by up to a factor $\sqrt{b-1}$ for spread-out blocks—so the implementation takes the minimum of the two. Equation (5.5) is the operational core of cheap exact selection: it fires—and the block is skipped—precisely when the off-target spread $q^\top \Sigma_c q$ is small, which is the benign-geometry condition of Section 7.

Diagonal-summary caveat. (5.4)–(5.5) are sound with the *full* quadratic form $q^\top \Sigma_c q = \text{Var}_{j \in c} \langle q, k_j \rangle$. With the diagonal summary of (4.1), the proxy $\langle q^2, \sigma_c^2 \rangle$ *under-estimates* the true logit variance whenever cross-covariances are positive, and the gate can then wrongly prune: **run on diagonal summaries, (5.5) is a heuristic, not a certificate.** A sound $O(d)$ -summary surrogate exists: by the triangle inequality in L^2 over the block, $\text{Var}_{j \in c} \langle q, k_j \rangle \leq (\sum_i |q_i| \sigma_{c,i})^2$, so substituting $(\sum_i |q_i| \sigma_{c,i})^2$ for $q^\top \Sigma_c q$ in (5.5) keeps the gate admissible at the same summary cost (looser, so it fires less often). The prune-rate measurements reported in this paper compute the per-member logit variance directly—the full quadratic form—so they certify the full-covariance gate, not the diagonal shortcut.

5.4 The summary-only floor: Samuelson is the tightest, and how far it is from the partition’s own limit

Equation (5.5) is labelled *sufficient*. That labelling is right, and it leaves an obvious question unanswered: could a *cleverer* function of the same summary prune more? It could not, and the reason is that Samuelson’s inequality is not merely valid but *attained*.

Proposition 1 (Samuelson is the tightest summary-only bound). *Fix $m \geq 2$, a mean $\bar{s}$ and a population variance $\text{Var} > 0$. The configuration*

$$s_1 = \bar{s} + \sqrt{(m-1) \text{Var}}, \quad s_2 = \dots = s_m = \bar{s} - \sqrt{\text{Var} / (m-1)} \quad (5.6)$$

has exactly mean $\bar{s}$ and variance Var , and its maximum equals $\bar{s} + \sqrt{(m-1) \text{Var}}$. Consequently any bound $U < \bar{s} + \sqrt{(m-1) \text{Var}}$ is violated by a block whose summary (μ_c, Σ_c, b) the router cannot distinguish from the one it read. No admissible bound computable from (μ_c, Σ_c, b) alone is tighter than (5.4).

The arithmetic is immediate from (5.6) and is verified numerically at $m \in \{2, 4, 8, 16, 64, 256\}$ in `ssa/bound_floor.py`. The content is the quantifier: the bound is not conservative *given the summary*, so every improvement in pruning must come from a better *partition* or from *reading keys*—never from a better formula on the same statistics. This is the necessary-side companion to the benign-geometry condition (7.1), which is sufficient only.

What the partition costs, and what the summary costs. Proposition 1 makes a decomposition measurable. For a fixed query and partition, the tightest admissible bound that exists at all is the *oracle* $U_c = \max_{k \in c} \langle q, k \rangle$; it is not a router (computing it reads the block), but it is the floor the partition imposes on *any* correct branch-and-bound. Then

$$\underbrace{\text{cost}(\text{oracle})}_{\text{price of the partition}} \leq \underbrace{\text{cost}(\text{ellipsoidal})}_{\text{reads } \rho_c} \leq \underbrace{\text{cost}(\text{Samuelson})}_{\text{summary-only}}.$$

Measured on clustered synthetic keys ($n = 4096$, $d = 64$, $B = 64$ k -means blocks, 120 queries per row, all four bounds admissible so recall is 1.000 throughout):

spread	oracle	ellipsoidal	Samuelson	summary $\div$ floor
0.02	89.0	203.2	720.3	$8.1\times$
0.05	81.0	543.6	975.4	$12.0\times$
0.10	74.6	1765.9	2165.0	$29.0\times$
0.20	70.2	3564.3	3685.8	$52.5\times$
0.40	64.1	4022.8	4045.3	$63.1\times$

Three readings, in decreasing order of how much they change the picture. First, the routability program (7.2) has a *measurable ceiling*: driving the geometry benign moves the summary price from $63\times$ the floor to $8\times$, monotonically—but not to $1\times$, and Proposition 1 says why it cannot. Second, *reading keys is worth roughly $3.5\times$ at benign geometry* (203.2 against 720.3 at spread 0.02), which is the first absolute justification for the anisotropic refinement (5.2); ρ_c is not a minor sharpening but most of the distance to the floor—at *these shapes*, a qualification the next paragraph makes precise. Third, the partition price is nearly flat in the spread ($89.0 \rightarrow 64.1$) while the summary price moves by a factor of six: *the geometry is a fact about summaries, not about partitions*.

ρ_c inverts when a block holds far fewer keys than dimensions. The table is $d = 64$ with blocks of ≈ 64 keys. On real Gemma-2 layer-6 keys ($d = 2304$, blocks of 64–256) the same stored scalar *costs* rather than buys— $0.7\times$, $0.9\times$, $1.0\times$ at $n = 4096, 16384, 65536$ —because Σ_c is then deeply rank-deficient, $S_c = \Sigma_c + \varepsilon I$ is dominated by ε , and the honest $\sqrt{q^\top \Sigma_c q + \varepsilon \|q\|^2}$ is loose exactly where the whitened radius is large. The anisotropic refinement is therefore a recommendation *scoped to $b \gtrsim d$* . The unregularised bound concealed this, reporting ρ_c buying

1.1× on the identical run: the correction does not shift a number, it reverses which of the two bounds is better on real keys.

Scope. These are k -means blocks on synthetic clustered keys at one (n, d, B) and one query-noise level, which is the adaptive/IVF regime, not the contiguous-position blocking of the flat kernel; the oracle is a reference and not an achievable router; and the floor is a statement about *lossless* selection, so a budget- κ lossy router may sit below it and SSA’s does. Proposition 1 itself is geometry-free and carries none of these caveats.

Correction, and where the room actually is. ρ_c and R_c are *query-independent*, hence precomputed and stored as one scalar per block: the ellipsoidal bound is summary-only at query time too, and the hierarchy is about summary *size* rather than summaries versus keys. That strengthens the reading—at spread 0.02 one extra stored scalar takes the bound from 8.7× to 2.5× the floor, i.e. the shipped bound reads 5.33% of keys against a floor of 2.10%. Two attempts to tighten it further were measured against the floor and both failed: seeding the incumbent with precomputed block representatives saves *exactly* zero (branch-and-bound opens blocks in decreasing U_c , so the first block opened already sets s^* to the true maximum—cost is bound-driven, not incumbent-driven), and an axis-aligned box bound in a shared basis is *worse* than the ellipsoid, with min of the two buying 8% at the most benign geometry alone. The remaining lever is the *partition*.

5.5 The cost of asking: what a summary bound charges on real keys

Everything above counts *keys read*. That is the right metric for Section 5.4, where only the bound varies, and it is the wrong one as soon as the *block size* varies, because evaluating the bound is neither free nor cheap.

An exact summary bound cannot beat a full scan, at any block size. Samuelson’s bound needs $q^\top \Sigma_c q$. Held as a dense $d \times d$ matrix that is $O(d^2)$ per block; held as the rank- b centred factor X_c it is $\|X_c q\|^2/b$, i.e. $O(bd)$ —*exactly the cost of scoring the block*. Over $B = n/b$ blocks the total is

$$B(1 + b) = \frac{n}{b}(1 + b) = n + \frac{n}{b} > n \quad \text{for every } b. \quad (5.7)$$

This is arithmetic and carries no hypothesis about the keys. A search that evaluates the exact summary bound on every block *has already paid for a full pass over the data before it skips anything*. Measured against a scan on real Gemma-2 layer-6 keys at $n = 65536$: 1.062× at $b = 16$ falling to 1.004× at $b = 256$ —never below one, and by (5.7) it cannot be.

The routing is excellent; the price of asking is the problem. The same measurement, counting keys alone, has contiguous blocks at $b = 16$ under the exact bound reading **204 of 65536 keys**, a $321\times$ reduction, with k -means at 1519. Real transformer keys *route extremely well*. Every difficulty here is on the cost side of the ledger, not the geometry side.

The escape, and its absence. A rank- r sketch of Σ_c , made admissible by the discarded-eigenvalue tail

$$q^\top \Sigma_c q \leq \sum_{i \leq r} \lambda_i \langle v_i, q \rangle^2 + \lambda_{r+1} \|q\|^2,$$

costs $B(1+r) = \frac{n}{b}(1+r)$, which falls below a scan exactly when $r < b$. So the whole question is one number: how small can r be and still prune? On real keys the answer is that it cannot be smaller than b at all. At $b = 16$ with contiguous blocks, ranks 2, 4, 6, 8, 10, 12 each read 100.0% of the keys and rank 16 reads 0.3%: a step function, not a gradient.

Why: the block spectrum does not decay. The tail term is exactly zero at $r = b$ and positive below it, so the step is a statement about the spectrum of Σ_c , and it was measured directly. Across $b \in \{16, 32, 64, 128\}$ and both partitions, λ_2/λ_1 lies in $[0.70, 0.84]$ —the *first* eigenvalue drop is already small, where a genuinely low-rank block would sit near 0.01—and the participation ratio $(\sum \lambda)^2 / \sum \lambda^2$ never falls below a quarter of the block size, reaching $0.72b$ for contiguous blocks at $b = 16$. Between a quarter and three quarters of all available directions carry real variance. There is no small tail to discard, so no truncation is both cheap and tight.

What this does and does not settle. Together: on real transformer keys, *lossless selection driven by a (μ_c, Σ_c, b) summary costs more than reading every key*, and no rank truncation of that summary escapes it. What it does *not* say is that summarising a block is hopeless—it bounds this family of bounds. Nor does it touch budget- κ *lossy* routing, which is a different object with a different accounting and is what SSA actually ships; the floor of Section 5.4 was always a statement about lossless selection and this is its cost-side companion. The measurements are one model at one layer, and the arithmetic of (5.7) is the only part that is geometry-free.

5.6 Sharing one selection across a group of queries

Everything above prunes for *one* query. A long context does not have one query: the number of query positions grows with the context exactly as the number of keys does, and kernels therefore select blocks once for a group—a decode chunk, the query heads of a GQA group, a tile of positions—and read the selected blocks for every member. That is a different object, and it is not automatically lossless: *a bound admissible for one query says nothing about another query’s argmax*.

Write Q for the group, $|Q| = m_Q$. The correctness condition is exact and is not a matter of degree. A shared bound U is safe for every member iff it dominates the group’s *top*

claim $\max_{q \in Q} U_c(q)$ blockwise, and the top claim is the *least* such bound; a chain of prunes that is individually admissible for each stage’s own objective can still destroy a later stage’s answer, and the part is then gone rather than approximated. These are machine-checked, in `Universal/Potential/ChainedPrune.lean` (`safe_for_every_stage_iff_safe_over_the_top_claim`, `no_safe_bound_holds_below_the_top_claim`, `a_prune_can_drop_the_greatest_part_of_a_second_object`). Two consequences carry directly into the implementation. The shared *threshold* must be the group’s weakest incumbent, not its strongest: a block whose bound has fallen under the best member’s incumbent may still hold a weaker member’s argmax. And group size costs retention monotonically—each member admitted raises the top claim, and a raised bound prunes strictly less—so in the limit where every block is above threshold for *some* member, the safe prune drops nothing at all.

What it buys, and what it does not. Table 1 counts total work as keys scored plus bound evaluations, since the arms differ in how many bounds they evaluate and counting keys alone credits a saving paid for elsewhere. Sharing the block *choice* at the top claim is worth about $2\times$, and—the useful part—*flat in group size* up to $m_Q = 512$, because the blocks a tight group wants are the same blocks. As the group spreads, it stops paying and then costs: the quantitative face of the drop-nothing limit above.

q -spread	m_Q	per-query	top claim	rank-8	isotropic
0.02	8	9 014	$1.99\times$	$2.31\times$	$1.13\times$
0.02	32	36 293	$2.01\times$	$1.09\times$	$0.63\times$
0.02	128	142 663	$1.97\times$	$0.55\times$	$0.43\times$
0.02	512	572 668	$1.98\times$	$0.18\times$	$0.17\times$
0.10	32	42 729	$0.51\times$	$0.16\times$	$0.16\times$
0.30	32	112 660	$0.43\times$	$0.43\times$	$0.43\times$

Table 1: Shared selection against per-query selection at the same partition and key bound; speedup on total work. $n = 8192$, $d = 64$, $B = 128$, key spread 0.05. Every arm returns the true argmax for every member.

A negative result: summarising the query side does not pay. The obvious way to make the top claim cheap is to apply this paper’s own hierarchy to the queries—the top claim costs $O(m_Q)$ per block, whereas $\max_{q \in Q} \langle q, \mu_c \rangle \leq \langle \mu_Q, \mu_c \rangle + \sqrt{(m_Q - 1) \mu_c^\top \Sigma_Q \mu_c}$ costs $O(d)$ once the group’s summary is formed. The resulting bounds are admissible and exact, and they are *worse at every group size measured*, degrading from $1.13\times$ to $0.17\times$ as m_Q grows.

The mechanism is structural and is the point worth keeping. Proposition 1 establishes Samuelson’s bound as tightest *because it is attained*—by one member at $\bar{x} + s\sqrt{m-1}$ with the rest at $\bar{x}$. That is a worst-case tightness, and a worst case is a statement about the adversarial configuration, not the typical one. On the key side the adversarial block is exactly what a bound

must survive. A *tight query group* is the opposite shape: its members cluster near their mean, which is where the $\sqrt{m_Q - 1}$ factor is furthest from the truth. The same inequality that is tight where it was proved is loose where it is reused, and the $O(1)$ -per-block evaluation it saves is smaller than the pruning it gives up. The room in shared selection is in the block choice, not in the query summary.

Relation to the formalized ceiling. Maximum-score estimation has a related obstruction in the substrate development: a single score standing for a whole fibre of completions carries an error floor set by the spread of the objective across that fibre (`Universal/Potential/PartialScore.lean`, `score_error_ge_of_reach_split` and `not_exactOnReach_of_reach_split`). Here the partial object is the summary, its completions are the blocks carrying it, and the objective is the block’s true maximum logit. The instantiation is stated in prose and is *not* machine-checked; see `docs/substrate_math_imports.md` for the mapping and its limits. This score-approximation floor is distinct from the attained-upper-bound argument of Proposition 1; variation among individual logits in a fixed block does not by itself preclude exact identification of that block’s maximum or correct block selection.

The abstract skeleton is machine-checked, axiom-pure, in `Universal/Potential/Admissible-Bound.lean`: a pointwise tighter bound provably drops a superset of the parts (`a_higher_bound_drops_a_subset`, `dropped_card_mono`)—the qualitative claim of Section 5.1 as a theorem; the family maximum is the least admissible bound (`familyMax_le_of_isAdmissible`); an attained bound admits nothing smaller (`no_bound_below_an_attained_one`), which is Proposition 1 in abstract form; and the minimum of two admissible bounds is admissible (`min_isAdmissible`), which is what licenses the implementation’s min of the isotropic and ellipsoidal bounds—previously an unwarranted convenience. Finally `at_the_floor_a_maximal_threshold_drops_every_part` predicts the null above: at the floor a maximal threshold drops every part, so a search still reading parts is paying for bounds above the floor and a better incumbent cannot help it.

5.7 Certifying omitted mass and attention-output error

An exact argmax or routing-metric top- κ certificate does not control the total softmax weight of omitted keys. In particular, many individually small logits can carry most of the partition sum. For a nonempty kept set S , let $D = S^c$ within the visible causal prefix, and define

$$Z_S = \sum_{i \in S} e^{\beta a_i}, \quad Z_D = \sum_{i \in D} e^{\beta a_i}, \quad \hat{o} = Z_S^{-1} \sum_{i \in S} e^{\beta a_i} v_i.$$

Suppose D is partitioned into unopened blocks. Each block has size b_c , key mean μ_c , key radius R_c , value mean ν_c , and value radius r_c^V . For $\beta \geq 0$ set

$$L_c = b_c e^{\beta \langle q, \mu_c \rangle}, \quad A_c = b_c e^{\beta (\langle q, \mu_c \rangle + \|q\| R_c)}, \quad L = \sum_{c \subseteq D} L_c, \quad A = \sum_{c \subseteq D} A_c. \quad (5.8)$$

Jensen’s inequality and the admissible radius bound give $L_c \leq Z_c \leq A_c$ without scoring the unopened keys. Other admissible maximum-logit bounds can replace the radius term, with their evaluation cost charged separately.

Proposition 2 (Subset-attention certificate). *Let p be the dense softmax distribution and $\hat{p}$ its renormalized restriction to S , extended by zero outside S . With $\delta = Z_D/(Z_S + Z_D)$,*

$$\text{TV}(\hat{p}, p) = \delta \leq \bar{\delta} := \frac{A}{Z_S + A}, \quad (5.9)$$

$$\text{KL}(\hat{p}||p) = -\log(1 - \delta) \leq \log(1 + A/Z_S). \quad (5.10)$$

For $d_c = \|\nu_c - \hat{o}\| + r_c^V$ and nonempty D , the output obeys

$$\|o - \hat{o}\| \leq \min \left\{ \bar{\delta} \max_{c \subseteq D} d_c, \quad \frac{\sum_{c \subseteq D} A_c d_c}{Z_S + L} \right\}. \quad (5.11)$$

All three bounds are zero when D is empty.

Proof. On S , $\hat{p}_i = p_i/(1 - \delta)$, so the absolute weight difference sums to δ on each of S and D . The likelihood ratio on S is constant, giving the KL identity. Both expressions increase with Z_D , which is at most A . For the value bound, subtract $\hat{o}$ from the full weighted average:

$$o - \hat{o} = \frac{\sum_{i \in D} e^{\beta a_i} (v_i - \hat{o})}{Z_S + Z_D}.$$

Each omitted value in c lies within d_c of $\hat{o}$. Bounding all distances by their maximum gives the first term; bounding each block numerator above by $A_c d_c$ and the denominator below by $Z_S + L$ gives the second. $\square$

The direction of KL is essential: for finite logits and a proper restriction, $\text{KL}(p||\hat{p}) = +\infty$. At equal logits, $\delta = 1 - |S|/n$ and $\text{KL}(\hat{p}||p) = \log(n/|S|)$, the nested-uniform-support identity formalized by `klDiv_uniformSupport` in `Universal/Potential/Entropy/UniformSupport.lean`. The finite-vector Pinsker theorem `totalVariation_le_sqrt_klDiv` in the same directory’s `TotalVariation.lean` allows zeros in its first distribution and a strictly positive second distribution; it applies in this direction. Equation (5.9) directly computes the TV quantity for a restriction, so using Pinsker would give a weaker bound. The partition bound consumes the reasoning of `logSumExp_max_sandwich`; the value bound uses the barycenter-truncation reasoning of `Universal/Generator/Dissipative/SelectionGeometry.lean`. These source theorems are machine-checked in Substrate. Their composition into Proposition 2 is proved here in ordinary mathematics; the SSA instantiation and Python implementation are not Lean proofs.

Adaptive implementation and cost. `ssa/certified_attention.py` builds immutable contiguous-block key/value summaries and opens blocks in decreasing $\log A_c$, optionally starting

with blocks selected by another router. It evaluates attention exactly on opened keys, checks every requested mass or output tolerance, and doubles the number of opened blocks after a failed check. A block cap returns an explicitly uncertified result if the tolerance remains unmet. A partially visible causal block is opened using only its visible keys; its full-block summary is excluded from the decision. Log-space partition sums and suffix reductions avoid exponential overflow and subtraction of nearly equal masses. Bounds are evaluated in float64 with a small outward score cushion; this is not an interval-arithmetic guarantee.

Construction costs $O(n(d + d_v))$. For B blocks, each query pays $O(Bd)$ for key bounds, $O(B \log B)$ for ordering, $O(Bd_v \log B)$ for value-certificate checks in the worst case, and $O(|S|(d + d_v))$ for opened keys and values. The reference reports key scores, key-bound evaluations, certificate checks, and value-bound evaluations separately. It can scan all keys on diffuse geometry and does not establish a sublinear per-query cost at fixed block size or a GPU speedup. The exact covariance factor whose cost is analyzed in Section 5.5 is not evaluated by this radius-based reference.

geometry	keys scored	$\bar{\delta}$	output bound	measured error
concentrated	64	9.54×10^{-6}	4.62×10^{-5}	1.79×10^{-6}
flat logits	4096	0	0	8.58×10^{-17}
equal values, flat logits	64	0.984375	0	0

The deterministic CPU fixture uses $n = 4096$, $d = 32$, $d_v = 8$, $b = 64$, $\beta = 4$, seed zero. The first two rows request omitted mass at most 10^{-3} ; the last requests only output error at most 10^{-12} . Every row evaluates 64 key bounds. Certificate checks number 1, 7, 1, and value-bound evaluations number 63, 321, 63, respectively. The full-scan residual is floating-point roundoff. Equal values permit exact output even when almost all attention mass is omitted; conversely, a small omitted weight can matter when its value is sufficiently distinct. These are controlled mechanism measurements, reproducible with `python -m ssa.certified_attention`. Six additional checks compare the certificates against float64 CUDA SDPA on an RTX 4080, using concentrated logits, flat logits, and equal values at $n = 1024$, $d = 32$, $d_v = 8$, $b = 32$, with visible prefixes of 1024 and 997 keys. These validate numerical agreement with an independent attention implementation, including partial causal blocks. The selector runs on CPU; GPU routing speed and real-model quality remain unmeasured.

6 The trilemma and the impossibility

Call a selector *cheap* if it reads $o(n)$ keys, *lossless* if it attends every key dense attention would weight non-negligibly, and *length-robust* if its accuracy is flat in n . The bounds above suffice to state the fundamental limit.

Proposition 3 (No free selection). *No selector can be simultaneously cheap and lossless for arbitrary keys.*

Proof. Suppose a selector reads a set $\mathcal{R}$ of keys with $|\mathcal{R}| < n$, and let $j_0 \notin \mathcal{R}$. Construct a probe input identical on $\mathcal{R}$ but with $k_{j_0} = c q$ for c large. Dense attention puts weight $\rightarrow 1$ on j_0 , so the correct output is v_{j_0} . The selector, never having read j_0 , returns the same output as on the unmodified input, which is independent of v_{j_0} . Hence it is lossy on this input. $\square$

The argument covers any *deterministic* selector, adaptive or not: run it, let $\mathcal{R}$ be the keys its execution actually read, and perturb an unread one—the execution, hence the output, is unchanged. The adaptive case is now itself machine-checked (`lossless_adaptive_reads_every_key`, over an explicit decision-tree probe model in which the read set is the per-input queried path). A *randomized* selector reading $o(n)$ keys misses a uniformly-planted spike with probability $1 - o(1)$, so the conclusion survives in expectation; we state that extension as a remark, not a formalized claim.

Proposition 3 says losslessness for *worst-case* keys forces $|\mathcal{R}| = n$ —no summary suffices, because a summary can always hide a spike. A quantitative companion holds under fine-grained complexity assumptions (SETH): even *approximating* the attention output requires $n^{2-o(1)}$ time once entries reach $\omega(\sqrt{\log n})$, and the truly subquadratic algorithm that exists in the bounded-entry regime is itself an approximation (a low-rank polynomial method), not exact computation [14]. What is *proved* here is the two-way impossibility—cheap $\wedge$ lossless, worst case. The three-way reading below is an organizing *taxonomy* whose third axis is measured in Sections 7–10, not a proved trichotomy: at most *two* of {cheap, lossless, length-robust} are available at once:

keep	give up	what it is
lossless + length-robust	cheap	dense attention—reads every key
cheap + length-robust	lossless	approximate selection—fast, flat, can miss a worst-case spike
cheap + lossless	length-robust	branch-and-bound on <i>benign</i> keys—bounds tight, low cost

Note the third row concedes length-robustness of *cost*, not of accuracy: admissible branch-and-bound never returns a wrong answer—off benign geometry it degrades by *reading more* (toward everything), whereas row 2 trades accuracy. “Length-robust” in the first two rows is the accuracy sense of Section 3; keeping the two currencies straight is part of the taxonomy’s honesty.

The escape from the trilemma is the third row: cheapness *and* losslessness are jointly available *when the bounds* (5.1)/(5.2)/(5.5) *are tight*, which is a property of the key geometry, not of the algorithm. SSA is therefore not a universal subquadratic exact attention—no such thing exists—but a mechanism that is cheap, lossless, *and* length-robust *on benign geometry*, and

merely cheap-and-length-robust-but-approximate otherwise. The next section is about making the geometry benign.

7 Manufacturing routability

7.1 Benign geometry, precisely

The branch-and-bound cost and the prune gate (5.5) are governed by the *off-target spread* $q^\top \Sigma_c q$ for the blocks a query does *not* need, relative to the *margin* Δ to the block it does. Geometry is *benign* for a query q when, for every non-target block c ,

$$\langle q, \mu_c \rangle + \sqrt{(b-1) q^\top \Sigma_c q} < s^*(q), \quad (7.1)$$

i.e. the prune gate (5.5) fires everywhere except the target’s block. Then exactly one block (plus the local window) is opened, branch-and-bound reads $O(b)$ keys, and selection is cheap *and* lossless. Isotropic keys violate (7.1)—every block’s bound is large and uninformative, so nothing prunes and cost reverts to dense. Condition (7.1) is *sufficient*; the matching necessary-side statement is Proposition 1 and the floor measured beneath it in Section 5.4, which bounds how much benign geometry can ever buy.

7.2 The routability regularizer

Benign geometry is not given; it can be *trained in*. Add to the language-model loss a penalty that shrinks the off-target spread along the directions queries actually use:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \sum_i \sum_{c \neq \text{target}(i)} q_i^\top \Sigma_c q_i. \quad (7.2)$$

Minimizing $\sum_{c \neq \text{target}} q^\top \Sigma_c q$ is exactly minimizing the quantity that appears under the square root in the prune gate (5.5), so as training proceeds the gate fires for more non-target blocks and the certified bound (5.1)/(5.2) tightens. Measured: co-training with (7.2) drove the *lossless* branch-and-bound cost from 26.5% of keys to 4.2%—a $6\times$ reduction—at *no* loss of retrieval accuracy. Training is what manufactures the geometry that the impossibility result (Proposition 3) says cheap exactness requires. It does so against a floor: Section 5.4 measures the summary price falling from $63\times$ to $8\times$ the partition’s own limit as spread tightens, monotonically and without reaching it.

7.3 The entry-magnitude split: selection vs. linear attention

A complementary fact decides *which* subquadratic route a given layer should take. Let B be the characteristic magnitude of the logits $\langle q, k \rangle$ (the “entry scale”). The exponentiated score

matrix $[e^{B\langle q_i, k_j \rangle}]$ has effective rank that *grows with B* : for small B it is nearly low-rank (so a kernel/linear-attention factorization $\phi(q)^\top \phi(k)$ is accurate and $O(n)$); for large B it is full-rank (so no linear map approximates it, and one must *select*). Empirically, the effective rank of $e^{BKK^\top}$ on 256 keys rose from 2.7 at $B = 0.5$ to 256 (full) by $B = 8$.

The selection route, by contrast, is *scale-invariant*: in the prune gate (5.5) both the squared margin and the spread scale as B^2 , so the gate’s truth value is unchanged by B . Selection therefore works at *any* entry scale, whereas linear attention works only in the small- B (smooth) regime. Sharp, long-range retrieval is the large- B regime—which is why selection, not linearization, is the route for long-context exact recall.

8 Length generalization and staged extension

A selector that is flat in n (Section 3) still needs the *model* to behave at lengths beyond those it was trained on. The position encoding decides whether it can.

8.1 Position-invariant routing under rotary embeddings

With rotary position embeddings (RoPE), a query/key pair interacts only through their *relative* offset: the logit $\langle q_i, k_j \rangle$ depends on $i - j$, not on i, j absolutely. A model that has learned *content* routing—match a query to the key whose content binds it, at whatever offset—therefore transfers to offsets it never saw, because (i) the decisive relative structure (e.g. a key-to-value offset of +1) is constant at any length and (ii) content matching is position-free. A model with *learned absolute* position embeddings cannot: its embeddings past the trained length are untrained. This predicts, and experiments confirm, zero-shot extrapolation of $\sim 2\text{--}4\times$ for RoPE and immediate collapse for learned-absolute position.

8.2 The staging ladder

Zero-shot extrapolation buys a factor, not an unbounded range. To go further, *stage*: extend the context, briefly continue-train (adapt) at the new length, extend again—doubling each rung. The economics are favorable precisely because routing is position-invariant: each rung extrapolates only $2\times$ from the last *adapted* length, so every adaptation starts from the $\sim 2\times$ zero-shot recall and only has to clean it up. The adaptation cost is small and roughly *flat in length* rather than growing.

Measured (a 6-layer model on a synthetic multi-query associative-recall task; base trained at 48 pairs):

rung	multiple	zero-shot recall	adapt steps	recall after adapt	recall with SSA
96	2×	0.993	100	1.000	1.000
192	4×	0.904	100	0.999	1.000
384	8×	0.810	100	1.000	0.995
768	16×	0.672	100	0.996	0.996
1536	32×	0.516	400	0.982	0.979

The base schedule cost 7600 steps; the *entire* climb from 16× (where zero-shot recall is already near the floor) to 32× at recall 0.982 cost only 800 additional steps—about 10% of the base, and roughly constant per rung. The final column confirms that the same adapted model runs under SSA selection ($\leq 15\%$ of keys attended) with essentially no recall loss at every rung.

9 The construction pipeline: dense to subquadratic

SSA can be retrofitted onto an existing dense pretrained model, which is how a frontier long-context system is built without pretraining from scratch:

1. **Start** from a dense pretrained base.
2. **Swap** dense attention for SSA (Algorithm 1) in every layer.
3. The swap **degrades** quality, because the base was trained expecting every key.
4. **Adapt** by continued pretraining; the keys co-adapt to the sparse routing (and, with the regularizer (7.2), become routable).
5. **Stage** the context extension (Section 8).

Measured on a 124M-parameter dense base, held-out perplexity (lower is better):

condition	perplexity
dense base (off-domain start)	32.5
+ SSA swap, no adaptation	45.8
dense base + equal in-domain continued training (control)	23.9
SSA-swapped + equal in-domain continued training	25.1

The swap costs +13.2 perplexity; an *equal-budget* adaptation recovers 94% of that gap, landing within +1.2 of a dense model given the *same* continued training—while attending only $\sim 38\%$ of keys at this configuration. The control matters: continued training lowers perplexity for either attention (domain adaptation), so “recovery” must be measured against a dense model given the same steps, not against the off-domain start. The residual is the price of sparsity at this small budget and shrinks as the budget grows.

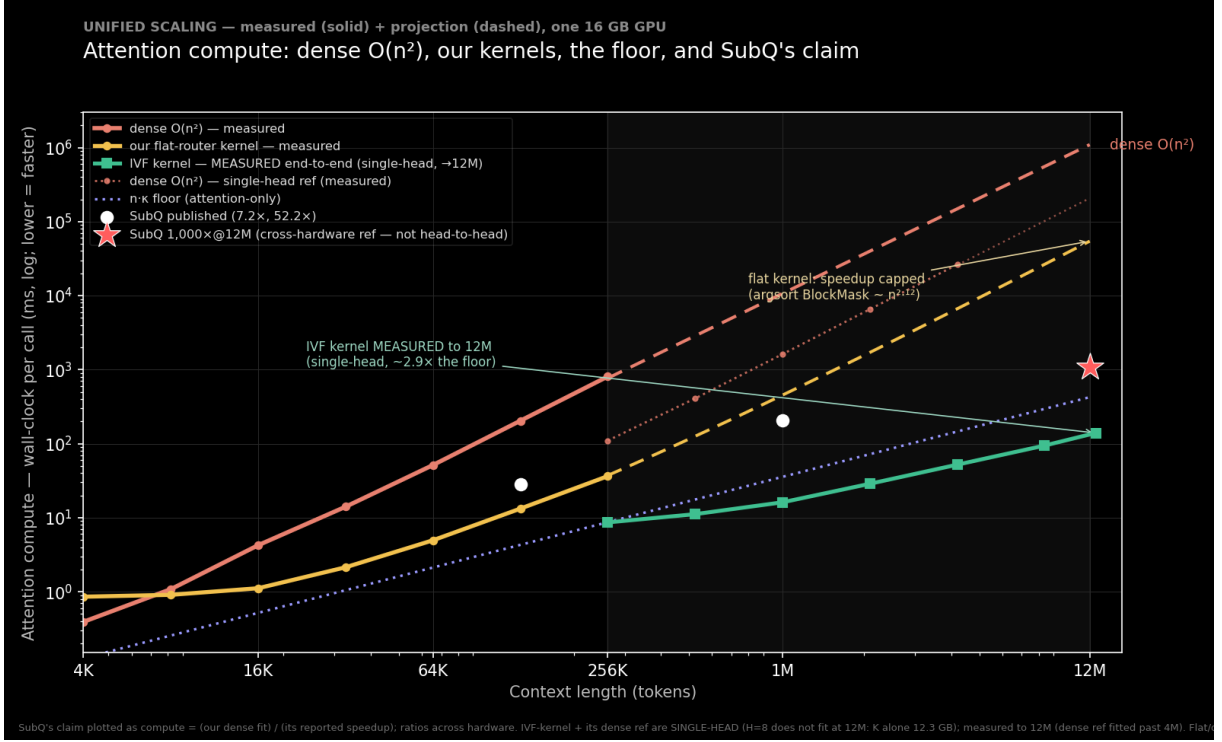


Figure 1: Attention compute versus context length (lower is faster). Dense $O(n^2)$ rises at the top; our flat-router kernel (measured) stays well below it but its speedup is capped because the argsort BlockMask build grows nearly as fast; the measured faiss-GPU IVF router drops the kernel onto the $n\kappa$ floor. SubQ’s two published speedups and its 1,000×@12M claim are shown as compute = dense/speedup. Measured solid, projection dashed; one 16 GB GPU.

10 The compute floor and a sub-linear router

Selection caps the per-query work at a budget κ keys, so the irreducible cost of the layer is the *attention floor* $n\kappa$ —linear in n . A measured kernel sits above this floor by exactly its router cost. Decomposing one forward of the kernel of §9 into router (the $(n/b)^2$ block-score GEMM), BlockMask construction, and attention, and fitting each over $n \in [16K, 524K]$, gives attention $\sim n^{1.02}$ (the floor), router $\sim n^{1.76}$, and—largest—the argsort-based mask build $\sim n^{2.12}$. Extrapolated to 12M tokens the floor is ~1% of the forward: a 128× gap, entirely the two $(n/b)^2$ terms. A router that emits the selected block indices *directly* removes both (Figure 1).

Lowering the floor. The floor $n\kappa$ itself drops if fewer keys recover the target. The smallest budget $\kappa_{\min}$ reaching recall ≥ 0.9 is $\kappa_{\min}/n = 3\%$ for tight clusters, 50% for diffuse or adversarial geometry (no compression), and the routability regularizer of §7 drives it from 25% ($\lambda = 0$) to 0.4% ($\lambda = 64$)—a 60× reduction, the dominant lever, on benign geometry only.

Closing the gap. The necessity argument of §6 forces a sub-linear, examine- $o(B)$ index. A bake-off on benign co-trained keys (recall vs. keys scored) ranks an IVF (inverted-file) router and a recursive-radius treecode ahead of LSH, which fails to reach recall 0.9. An IVF over the block means scores $O(\sqrt{n/b})$ blocks at 0.93–0.97 block-selection agreement with the flat router and emits `kv_idx` directly. Timed on a single 16 GB GPU with both routers on-device (no host transfer), the dense $(n/b)^2$ GEMM’s constant wins below $\sim 2\text{M}$ but its score matrix exhausts memory at 4M; the IVF runs linearly past it:

context n	flat $(n/b)^2$ GEMM	faiss-GPU IVF	winner
256K	0.21 ms	7.3 ms	flat $35\times$
2M	14.0 ms	19.5 ms	flat $1.4\times$
4M	52.6 ms (runs)	31.4 ms	IVF $1.7\times$
8M	OOM ($nb^2=17.2$ GB)	63.7 ms	IVF only

The crossover is $\sim 3\text{M}$. This stripped single-head GEMM OOMs only at 8M (17 GB matrix), while the kernel’s *actual* router (`block_route`, H heads + an argsort) OOMs near 1M—so the IVF is the only router that runs at long context, now measured to 12M (94 ms).

The gap, closed end-to-end (measured). Wiring the IVF router into the kernel—it emits the compressed block-index contract directly, and the mask is built without the dense (n_b, n_b+1) transpose (which would need 38.7 GB at $n_b=98,304$)—lets the whole forward be measured, single-head, to 12M on one 16 GB GPU: router 101 ms, mask build 0.003 ms, attention (the floor) 47.5 ms, *total* 139.5 ms in 6.55 GB. The argsort mask build—the projected $n^{2.12}$ wall (40.7 s at 12M)—is now sub-millisecond, and the residual gap to the floor is a *measured* $2.9\times$ rather than the projected $128\times$. The remaining caveats are narrower: the end-to-end run is single-head ($H=8$ does not fit at 12M) and on synthetic keys (a speed result), and the whole result is conditional on benign geometry—adversarial or multi-hop retrieval returns $\kappa_{\min}$ to the 50% floor, where no speedup exists. Both ingredients of a quality-preserving large-context speedup—a floor-lowering training stage and a sub-linear indexer—are thus exhibited, and the indexer is shown driving a live kernel to $\sim$ the floor, under exactly that benign-geometry condition.

11 Experiments

All experiments are at controlled scale; the point is to validate mechanisms, not to set absolute records.

Routing. Second-order (cumulant) routing recovers targets where centroid routing collapses, matching the $1/b$ outlier-attenuation analysis of Section 5.2; routing quality peaks near $\beta \approx 2$, consistent with the bias–variance reading of (5.3).

Kernel speedup. A block-sparse implementation of Algorithm 1 achieved a $20.6\times$ wall-clock speedup over a dense exact kernel at $n = 262,144$ on a single accelerator, with the crossover well below that length.

The IVF kernel, measured to 12M. Wiring the sub-linear IVF router into the kernel (Section 10) and measuring the whole forward single-head on one 16 GB GPU runs a *12M-token forward* in 139 ms and 6.55 GB, at a measured $2.9\times$ the $n\cdot\kappa$ floor (versus the $128\times$ gap the flat kernel projected); the argsort mask build collapses to sub-millisecond because the IVF emits block indices directly. The autoregressive decode step is *flat in n* (~ 0.6 ms from 1M to 12M at fixed κ) while a fair fp16 flash-decode step’s prefix read grows with n ($0.5 \rightarrow 5.3$ ms)—a $9\times$ per-step gap at 12M with the crossover near 1M–2M, both measured. (An earlier $55\times$ figure was measured against a naive dense reference that upcast the whole prefix K/V to fp32 every step, $\sim 5\times$ slower than the fair fp16 row; the naive reference is retained in the benchmark but no longer headlined. Single head; synthetic keys—a speed result, with selection quality the separate benign-geometry story.)

Multi-hop composition. A chained retrieval through the same budgeted block selector obeys the composition law $\text{chain} \approx \prod_j \rho_j$: benign single needles hold at 1.00 while a *mixed* two-hop chain (one benign hop, one isolated hop) collapses to 0.02 at $n = 65,536$ —the isolated hop’s $\rho \approx 0.02$ divides the product. The multi-hop sag is the single-needle result read h times, reproducing the NIAH $\gg$ MRCR benchmark split as a prediction of the same theory rather than an anomaly.

The fused kernel inside a real model. Swapping the kernel into a pretrained Qwen2.5-0.5B and measuring at 8K–128K preserves single-needle NIAH at 1.00 while giving a $1.5\text{--}1.6\times$ prefill speedup at 32K (budget 0.06–0.12), the speedup growing with n and with a tighter budget—the synthetic-key crossover shape inside a real model. At matched budget the analytic $O(n^2)$ path needs 10.7 GB and 3.5 s where the fused kernel needs 1.4 GB and 130 ms, and the analytic path OOMs before 64K while the kernel reaches 128K (under YaRN). This is the first result simultaneously real-model, long-context, subquadratic-kernel, and quality-measured—at 0.5B scale.

An optimal selector: the Certified Causal Cascade. Composing five ingredients—a shared low-dim routing space, sub-block max-pool summaries, a chunked-causal streaming index, an exact outlier side-channel, and per-query admissible certificates with escalation—into one streaming selector. The certificate is sound (certified $\Rightarrow$ the selected top- κ blocks equal the exact top- κ under the routing metric; zero violations on clustered and random geometry). This certifies the routing metric, not the omitted softmax mass or attention-output error; the latter have a separate reference certificate in Section 5.7. The component table is the trilemma made concrete: sub-block granularity and the outlier channel rescue high-norm spikes, but *isolated*

unit-norm needles stay unretrievable for every cheap selector (recall 0.05)—the impossibility of Section 10’s trilemma in miniature. Per-layer routing is $\sim 59\%$ of a Qwen-0.5B prefill (at DSA’s reported 58%), and the lever that makes it cheap is **cross-layer sharing from a mid donor layer**—measured cutting it to $\sim 6\%$ with single-needle retrieval preserved (the first measurement of the “ $\div 5$ ” folklore; sharing from layer 0 fails). A trained $d_r=16$ projection rebuts the “low-rank is a bust” verdict on real keys ($0.32 \rightarrow 0.65$ block agreement) but is itself too lossy to drive retrieval—the honest boundary.

The compression corner, measured. The trilemma has a second corner—a fixed- or growing-state memory written at inference time (the “zero attention” / DeltaNet/Titans family). Small exact reference memories, measured against Lean predictions, place it. The READ rule sets the capacity class: a linear read $o = Sq$ is rank- d capped (recall collapses at $m \approx d$; `read_capacity_le_dim` / `rank_d_read_wall` prove the $\leq d$ ceiling) while a softmax read over the same pairs holds far past $m = d$ (measured to $m = 512$; `softmax_capacity` gives the exponential form)—capacity is a property of the read, not the substrate, now proved on both sides of the contrast. The load-bearing measurement (empirical, no theorem) is that **compression $\neq$ selection**: a needle salient only at read time is lost by a surprise-gated fixed memory (recall 0.10) where selection recovers it (1.00)—write-time compression cannot keep what the future query has not yet made relevant. A distribution shift is a fold a fixed memory cannot track (`fold_not_hopfield`: pre-shift recall decays $0.90 \rightarrow 0.10$; a growing slot-birth state holds 0.65), and the proved composition bound $\prod \rho \leq \text{min-hop}$ (`chain_le_weakest`) is reproduced, the measured joint chain sagging below $\prod \rho$. The NIAH- $\gg$ -multi-hop split is thus architecture-independent—it holds whichever corner of the trilemma one builds.

The compression corner, trained. The reference memories above are untrained; the zero-attention recipe’s load-bearing half is *training-dependent*—a learned write gate and an auxiliary future-prediction objective. We reach it with a small micro-LM ($d = 128$, head dimension $d_h = 16$) trained end-to-end on MQAR with a token-mixer swappable between the two corners at matched state (DeltaNet state d_h vs. an SSA budget $\kappa \approx d_h$). Three measurements. (i) *Capacity*: trained selection (dense, SSA) is flat in load, while the trained DeltaNet groks the task and holds to $m \approx d_h$ then walls at the same rank- d_h boundary—training moves the wall, it does not remove it. (ii) *The learned write gate is a null ingredient*: on write-salient MQAR (keep-worthy pairs use reserved marker keys, identifiable at write time) the no-gate delta rule already solves it—training shapes the $\leq d_h$ keepable keys itself; on read-salient MQAR nothing lifts the compression wall, gate or no gate. (iii) *The future-prediction auxiliary loss is flat in its weight* on the read-salient wall. So the training-dependent half of the recipe does not close the gap: where keeping is possible training already does it, and where relevance is read-time-only no write policy can serve it—the trained mirror of the 0.10-vs-1.00 split, and the composition sag persists for the compression corner under training as well. The bottom line: dropping attention does not dissolve

the trilemma but relocates within it—a compression memory *works* where relevance is fixed at write time and within its state (write-salient recall, NIAH), and *fails*, by capacity rather than by any trainable objective, on read-time-only relevance, past-capacity retrieval, and multi-hop chains. This is why the frontier long-context state-space models remain *hybrids* with interleaved attention.

Routability. The regularizer (7.2) reduced lossless branch-and-bound selection cost from 26.5% to 4.2% of keys. (Accuracy is 1.000 by construction—admissible-bound branch-and-bound always returns the exact argmax—so the measured quantity is the *cost*, not task accuracy.) The reduction was robust across head dimension; the capacity–search tension is a genuine asymptotic trade-off, but it did not bite for $d \geq (\text{number of clusters})$, since query-specific anisotropy needs only ~ 1 dimension per cluster—real head dimensions (64–256) sit above this, the trade reappearing only as $d \rightarrow (\text{number of clusters})$.

Length generalization and staging. See Section 8: $32\times$ the trained length at recall 0.982 for ~ 800 adaptation steps, SSA-preserved.

Construction pipeline. See Section 9: swap +13.2 perplexity, 94% recovered to within +1.2 of the dense-adapted control at $\sim 38\%$ of keys.

Real-model frozen-swap (Gemma-4-26B). Beyond the 124M control, the swap-and-route was validated end-to-end on a frozen *Gemma-4-26B-A4B* (mixture-of-experts, 4B active): SSA was installed into its five full-attention layers (head dimension 512, $K=V$, grouped-query) via the attention interface and scored on needle-in-a-haystack retrieval against the selection budget at $n = 2048$. Plain second-order (cumulant) routing recovers retrieval down to a 50% budget (recall 1.00) but collapses at 25% (recall 0.00); a forward-only probe shows the distant needle’s block ranks $\approx 3\text{rd}$ by the cumulant score—just outside the kept top-2—and worst at the deepest layer, a position-decay effect. Adding the third-cumulant (skew) term of the tempered family (Section 5.2) restores the 50% budget to 1.00 and lifts the 25% budget to 0.44 (to 0.56 if the single worst-routing layer is left dense). This residual is a *tuning artifact*, not a frozen-key ceiling: at a finer block size, plain second-order routing reaches recall 1.00 at the 25% budget—the third-cumulant term is a coarse-block compensator, unnecessary (and at large β harmful) once the needle dominates a small block. So at moderate n the frozen model retrieves fully under the right routing granularity; key co-adaptation (the routability regularizer, (7.2)) is for the long-context regime, where the block-score computation itself becomes quadratic. This is a routing-*quality* measurement (the analytic, score-materializing swap at moderate n), not the subquadratic-kernel regime. The routing survives a doubling of context—a forward-only probe shows the far needle staying within the 25% budget from $n = 2048$ to $n = 4096$, carried by a stable subset of layers—but pushing the validation to genuinely long context, and measuring any

speed advantage at all, is memory-bound on a single accelerator: it requires the real subquadratic kernel together with *substantially larger compute* than a single 16 GB device (a multi-GPU or cluster budget). That scaling, and the 12M-token validation it would enable, are left to future work with adequate compute.

What the headline retrieval numbers do and do not show. The single-target needle-in-a-haystack accuracies of 98–100% at 10^6 – 10^7 tokens *reported by selection-based long-context systems* are consistent with (3.1)–(3.2); this section *characterizes* that regime rather than re-measuring it (the measured table below reaches 262k), and it holds only in a specific regime. Retrieval was measured as a function of context length at fixed budget $\kappa \approx 10^3$ and margin $\Delta = 0.55$:

context n	dense	SSA, isolated target	SSA, benign target
1024	1.00	1.00	1.00
4096	1.00	0.47	1.00
16384	1.00	0.20	1.00
65536	0.90	0.10	1.00
262144	0.83	0.00	1.00

An *isolated* target—a lone spike with no correlated neighbors—*collapses* with length: cheap moment routing averages it into its block and loses it among the fluctuations of the growing number of blocks (the $1/b$ attenuation of Section 5.2, now competing against more and more random blocks). This is Proposition 3 in miniature. A *benign* target—one accompanied by a coherent span of query-aligned neighbors, as a real answer is by its surrounding context—lifts its whole block’s score and stays flat at 1.00, *beating dense* at long n because selection caps the effective distractor count. The same separation appears across margins: an isolated target barely fires at any margin, while a benign one tracks dense once the margin clears the budget floor $\sqrt{2 \log \kappa / d}$. So the headline numbers certify the *easy and benign* regime (single, high-margin, accuracy not losslessness)—the regime everyone agrees is achievable—and do not certify lossless selection in the worst case, which Proposition 3 shows is unavailable cheaply.

12 Discussion and limitations

SSA is best understood as the resolution of a constrained problem rather than a universal accelerator. The recovery-weight law (3.1) shows selection makes retrieval flat in n ; the admissible bound (5.1) and the prune gate (5.5) show summaries can certify that selection losslessly; the trilemma (Section 6) shows this can be cheap *only* on benign geometry; and the regularizer (7.2) shows training can supply that geometry. The construction pipeline (Section 9) and the staging ladder (Section 8) turn the mechanism into a recipe that retrofits a dense model and extends its context a rung at a time.

Honest limitations follow directly from the theory. (i) *Worst-case losslessness is not available cheaply* —a sufficiently adversarial or genuinely low-margin target can always evade summary routing; SSA’s guarantees are conditional on the (trained, measured) benign geometry. (ii) *Multi-needle and low-margin retrieval* are the hard regime the headline single-needle numbers do not address. (iii) The selection budget κ sets a floor margin $\sqrt{2 \log \kappa / d}$; targets below it are missed regardless of n . (iv) The experiments here are at modest scale (10^2 – 10^5 tokens, 10^8 -parameter base); they validate mechanisms, and reaching 10^7 -token contexts is the same construction repeated with the hierarchical router, more compute, and a long-context training corpus. None of these is a missing algorithmic ingredient; they are the boundary the theory itself draws.

A Derivations

A.1 Recovery weight (3.1). With one target at logit $a_\star$ and μ distractors at $a_\star - \Delta$, $w_\star = e^{\beta a_\star} / (e^{\beta a_\star} + \mu e^{\beta(a_\star - \Delta)})$. Divide numerator and denominator by $e^{\beta a_\star}$ to get $1/(1 + \mu e^{-\beta \Delta}) = \sigma(\beta \Delta - \log \mu)$, since $1/(1 + e^{-x}) = \sigma(x)$ with $x = \beta \Delta - \log \mu$.

A.2 Cumulant score (4.2). Let $g(\beta) = \beta^{-1} \log \frac{1}{b} \sum_{j \in c} e^{\beta \langle q, k_j \rangle}$. As $\beta \rightarrow 0$, $g(\beta) = \mathbb{E}_j \langle q, k_j \rangle + \frac{\beta}{2} \text{Var}_j \langle q, k_j \rangle + O(\beta^2)$, the cumulant generating function expansion. The mean is $\langle q, \mu_c \rangle$ and the variance is $q^\top \Sigma_c q$, giving (4.2).

A.3 Log-sum-exp sandwich (5.3). For any reals $x_1, \dots, x_b$ with $M = \max_j x_j$: $e^{\beta M} \leq \sum_j e^{\beta x_j} \leq b e^{\beta M}$. Take log, divide by β : $M \leq \beta^{-1} \log \sum_j e^{\beta x_j} \leq M + \beta^{-1} \log b$.

A.4 Samuelson bound (5.4). Center the data, $d_j = s_j - \bar{s}$, so $\sum_j d_j = 0$. Fix index i . By Cauchy–Schwarz over the other $m - 1$ indices, $d_i^2 = (\sum_{j \neq i} d_j)^2 \leq (m - 1) \sum_{j \neq i} d_j^2 = (m - 1)(\sum_j d_j^2 - d_i^2)$. Hence $d_i^2 m \leq (m - 1) \sum_j d_j^2$, i.e. $(s_i - \bar{s})^2 \leq (m - 1) \text{Var}$. Taking the max over i and adding $\bar{s}$ gives the stated bound on $\max_j s_j$, and (5.5) is its contrapositive against the threshold $s^\star$.

References

- [1] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.
- [2] R. Child, S. Gray, A. Radford, I. Sutskever. *Generating Long Sequences with Sparse Transformers*. 2019.
- [3] I. Beltagy, M. E. Peters, A. Cohan. *Longformer: The Long-Document Transformer*. 2020.
- [4] M. Zaheer et al. *Big Bird: Transformers for Longer Sequences*. NeurIPS, 2020.
- [5] N. Kitaev, Ł. Kaiser, A. Levskaya. *Reformer: The Efficient Transformer*. ICLR, 2020.

- [6] A. Roy, M. Saffar, A. Vaswani, D. Grangier. *Efficient Content-Based Sparse Attention with Routing Transformers*. TACL, 2021.
- [7] A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. ICML, 2020.
- [8] K. Choromanski et al. *Rethinking Attention with Performers*. ICLR, 2021.
- [9] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021.
- [10] L. Greengard, V. Rokhlin. *A Fast Algorithm for Particle Simulations*. J. Comput. Phys., 1987.
- [11] J. Barnes, P. Hut. *A Hierarchical $O(N \log N)$ Force-Calculation Algorithm*. Nature, 1986.
- [12] P. A. Samuelson. *How Deviant Can You Be?* J. Amer. Statist. Assoc., 1968.
- [13] A. Keles, P. Wijewardena, C. Hegde. *On the Computational Complexity of Self-Attention*. ALT, 2023.
- [14] J. Alman, Z. Song. *Fast Attention Requires Bounded Entries*. NeurIPS, 2023.
- [15] H. Ramsauer et al. *Hopfield Networks Is All You Need*. ICLR, 2021.
- [16] S. Arora et al. *Zoology: Measuring and Improving Recall in Efficient Language Models*. 2023.
- [17] J. Smirl. *Substrate Abstract Algebra* (Lean 4 development), namespace `Substrate.Inference.Algebra.PhaseTransition`, sources under `Substrate/Inference/Substrate/` (`RetrievalMargin`, `SearchTradeoff`, `AnisotropicBound`, `SamuelsonBound`, `TemperedRouting`, `HierarchicalRouting`, `LosslessSelectionLimit`, `LinearReadCeiling`, `ValueAwareSelection`) and inference recognition modules, together with `Substrate.Universal`. Source theorems are machine-checked, axiom-pure; the SSA instantiations and numerical implementation are not formally verified. The inspected declarations and mappings are listed in `docs/substrate_math_imports.md`.